



Sunbeam Institute of Information Technology
Pune and Karad

Data structures and Algorithms

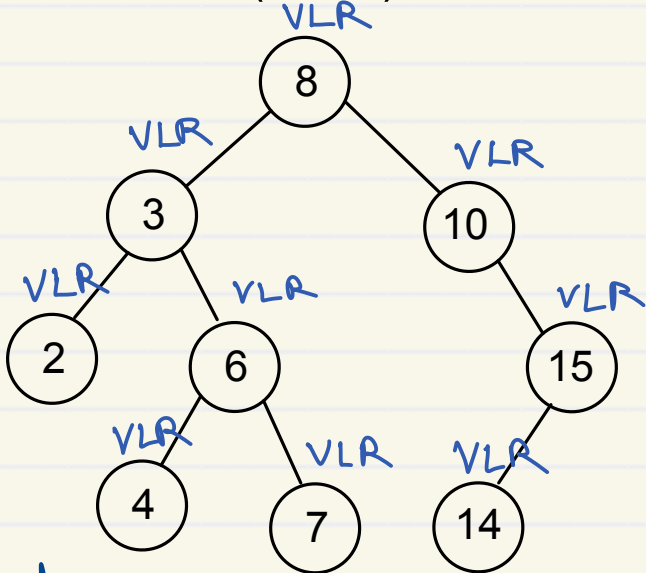
Trainer - Devendra Dhande

Email – devendra.dhande@sunbeaminfo.com

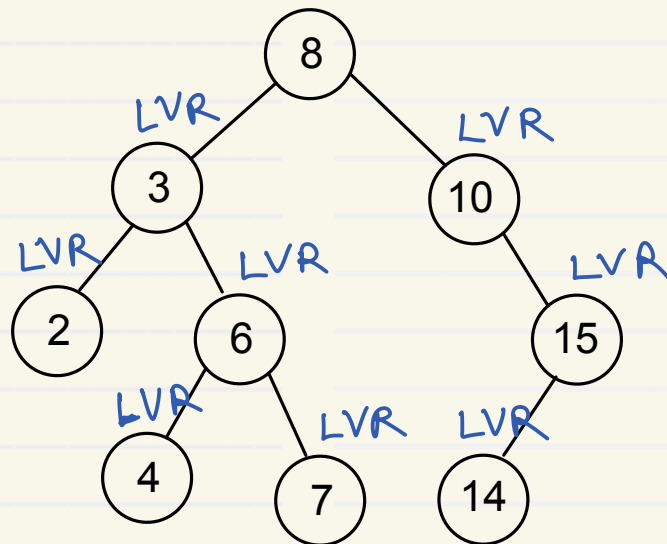
Tree Traversal Techniques

- The traversal algorithms can be implemented easily using recursion.
- Non recursive algorithms for implementing traversal needs stack to store node pointers.

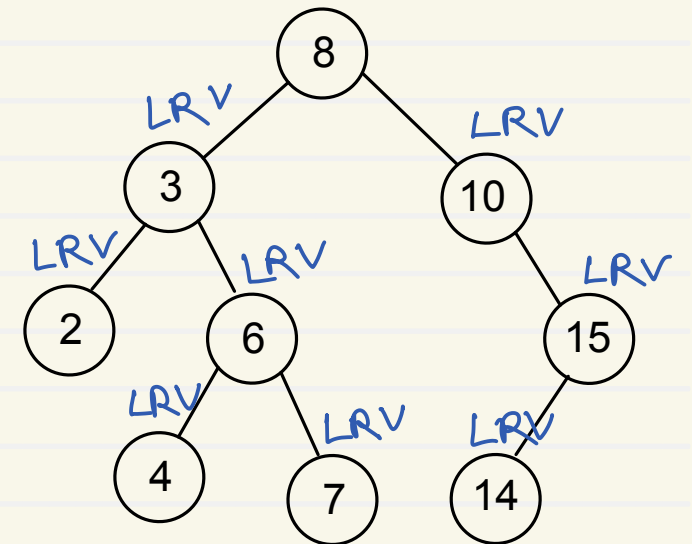
Preorder
(VLR)



Inorder
(LVR)

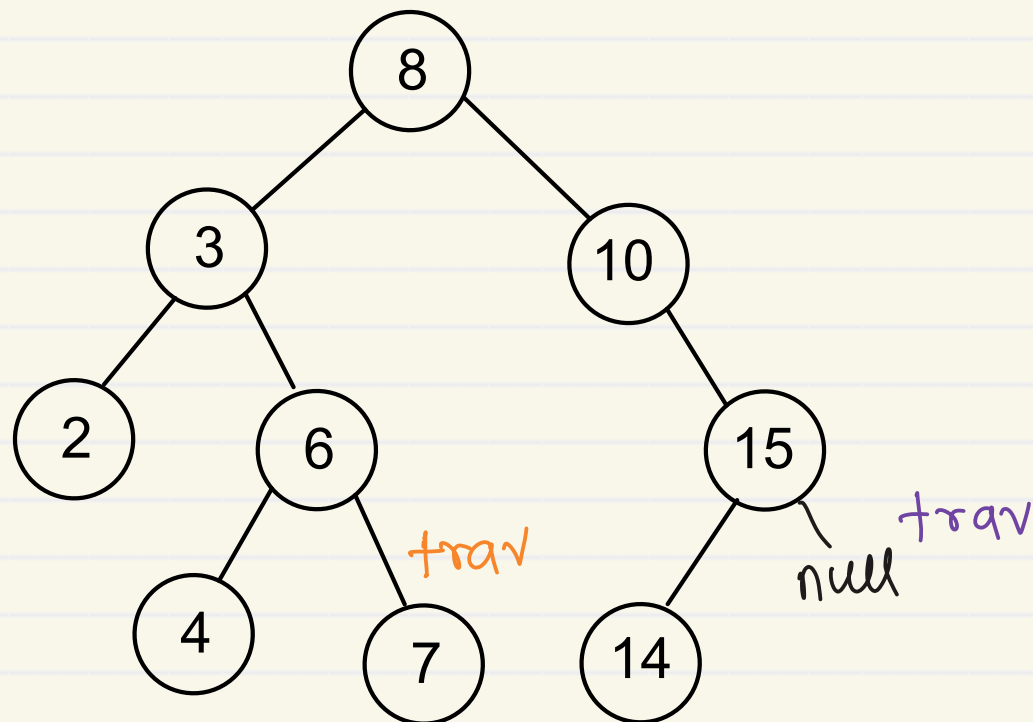


Postorder
(LRV)



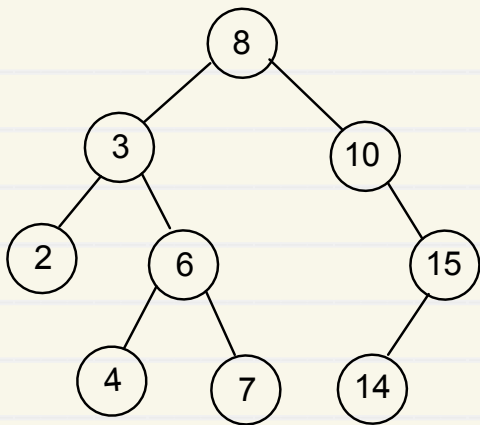
Preorder : 8, 3, 2, 6, 4, 7, 10, 15, 14 Inorder : 2, 3, 4, 6, 7, 8, 10, 14, 15 Post Order : 2, 4, 7, 6, 3, 14, 15, 10, 8

Binary Search Tree - Binary Search



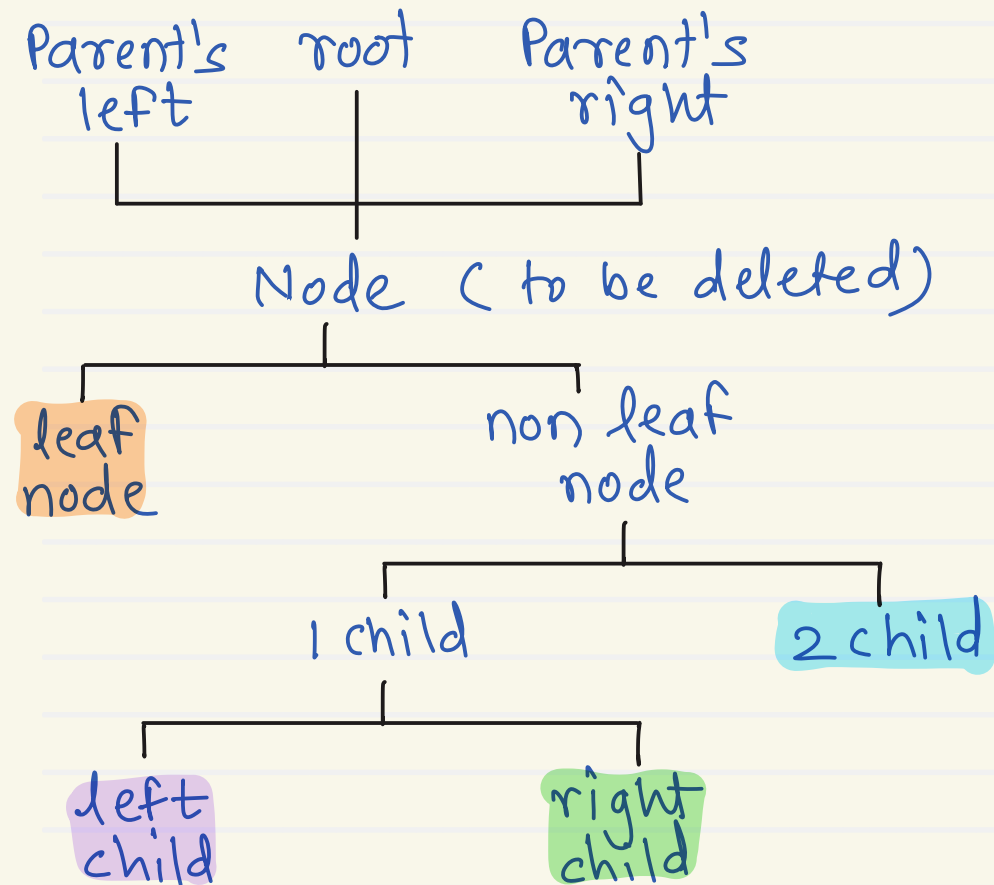
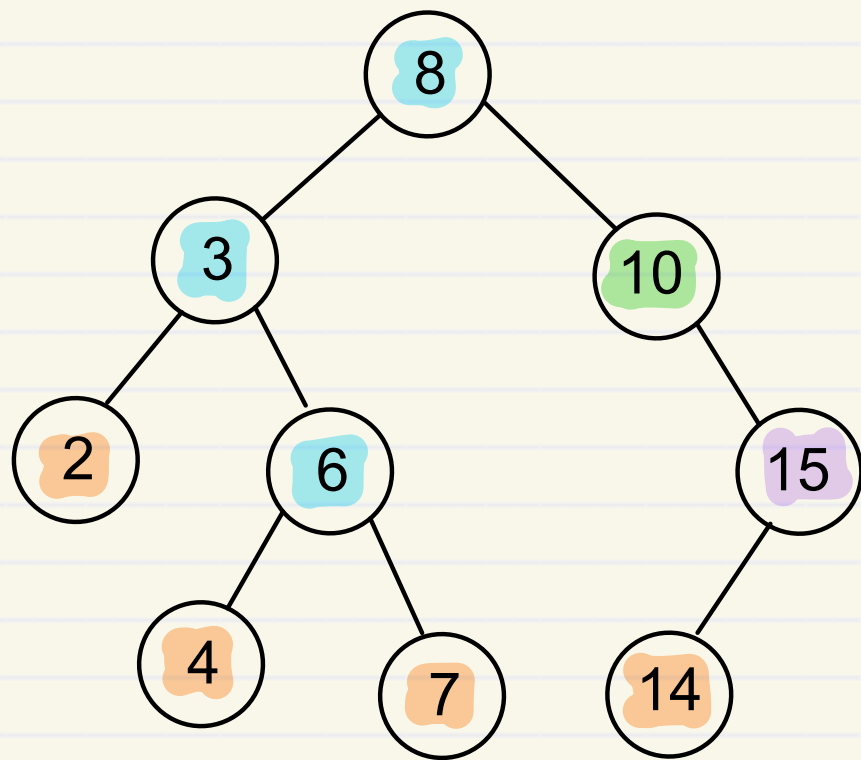
1. Start from root
2. If key is equal to current node data return current node
3. If key is less than current node data search key into left sub tree of current node
4. If key is greater than current node data search key into right sub tree of current node
5. Repeat step 2 to 4 till leaf node

key = 16				key = 7			
trav	data			trav	data		
8	8	>		8	8	<	
10	10	>		3	3	>	
15	15	>		6	6	>	
null				7	7	=	

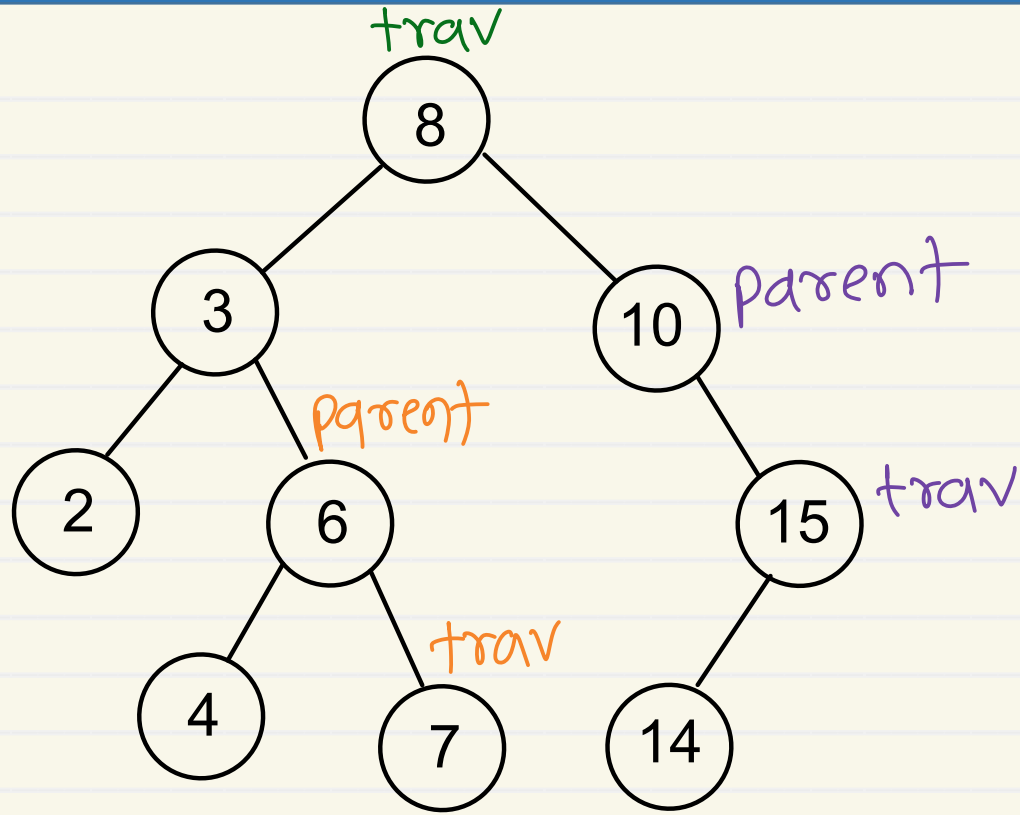


```
Node binarySearchRec(Node trav, int key) {  
    if (trav == null)  
        return null;  
    if (key == trav.data)  
        return trav;  
    if (key < trav.data)  
        return binarySearchRec(trav.left, key);  
    else  
        return binarySearchRec(trav.right, key);  
}
```

Binary Search Tree - Delete Node



Binary Search Tree - Binary Search with Parent



Key = 7

trav	parent
8	null
3	8
6	3
7	6

Key = 8

trav	parent
8	null

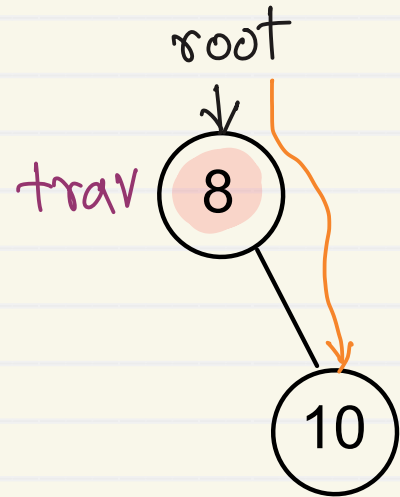
Key = 16

trav	parent
8	null
10	8
15	10
null	15

BST - Delete Single child node (Right child)

root
node

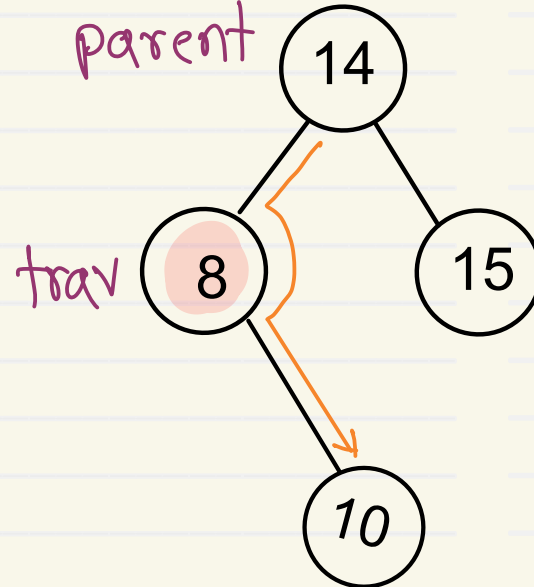
parent = null



if (trav == root)
root = trav.right

Parent's
left node

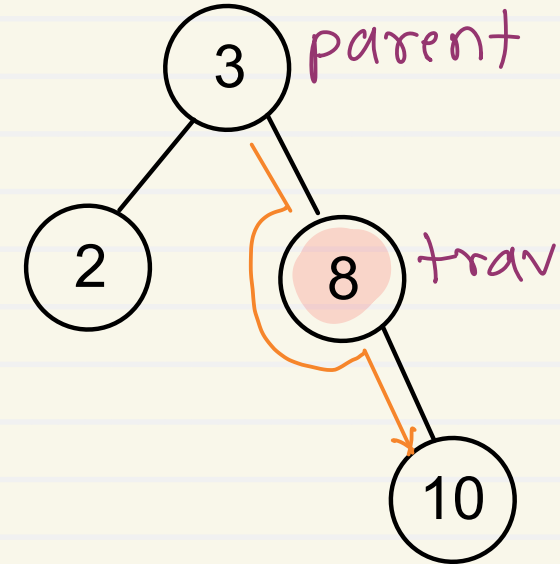
parent



if (trav == parent.left)
parent.left = trav.right

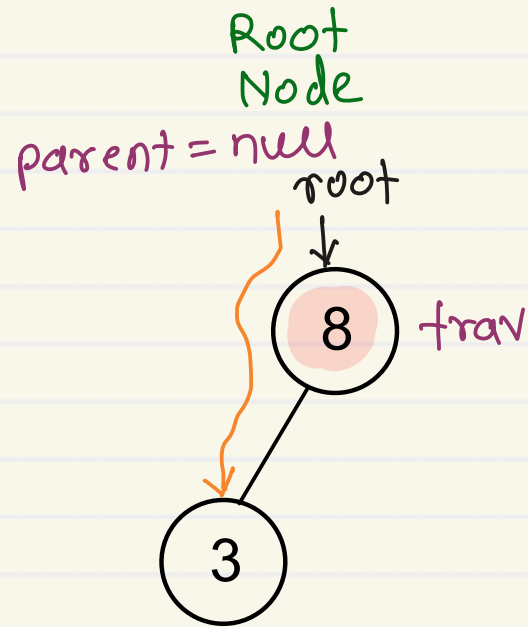
Parent's
right node

parent

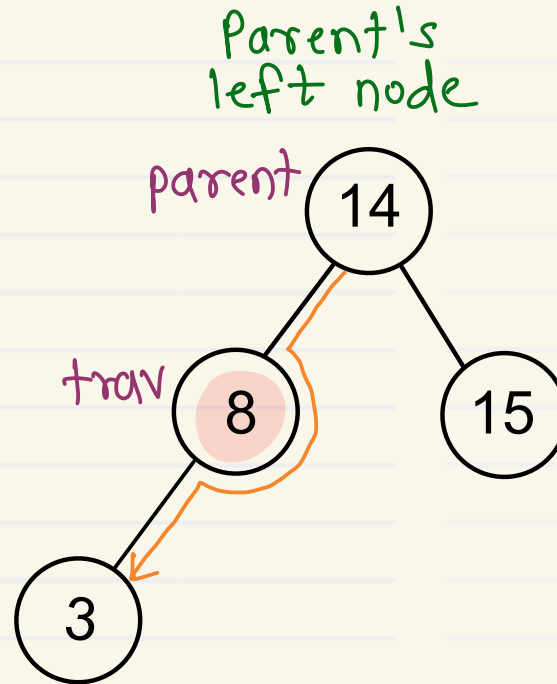


if (trav == parent.right)
parent.right = trav.right

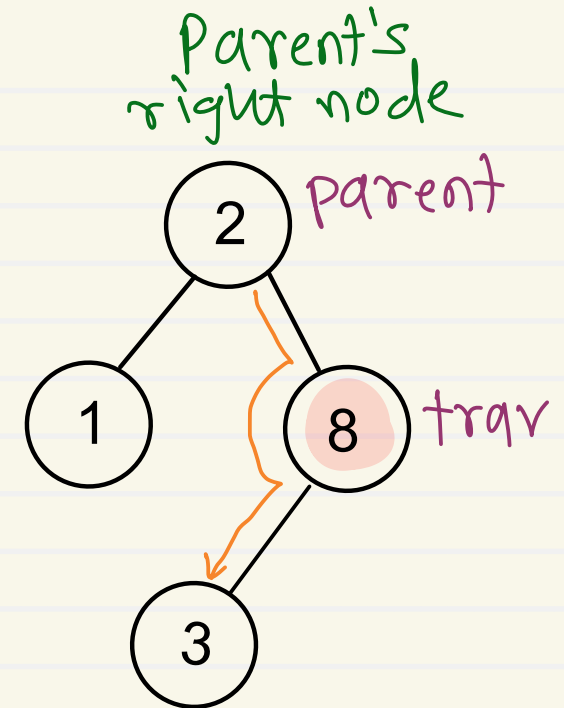
BST- Delete Single child node (Left child)



if (trav == root)
root = trav.left

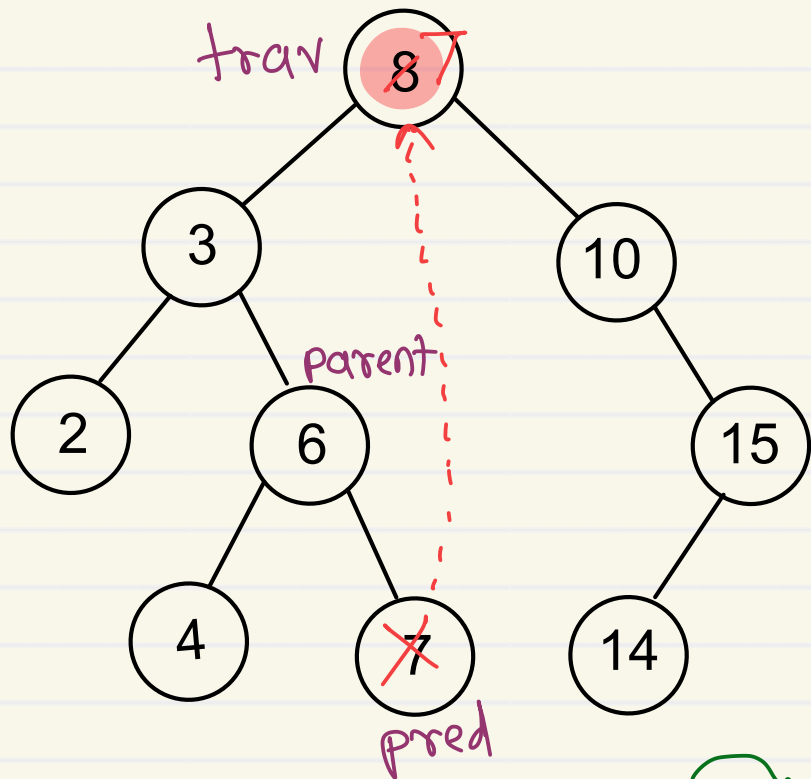


if (trav == parent.left)
parent.left = trav.left



if (trav == parent.right)
parent.right = trav.left;

BST - Delete Two childs node



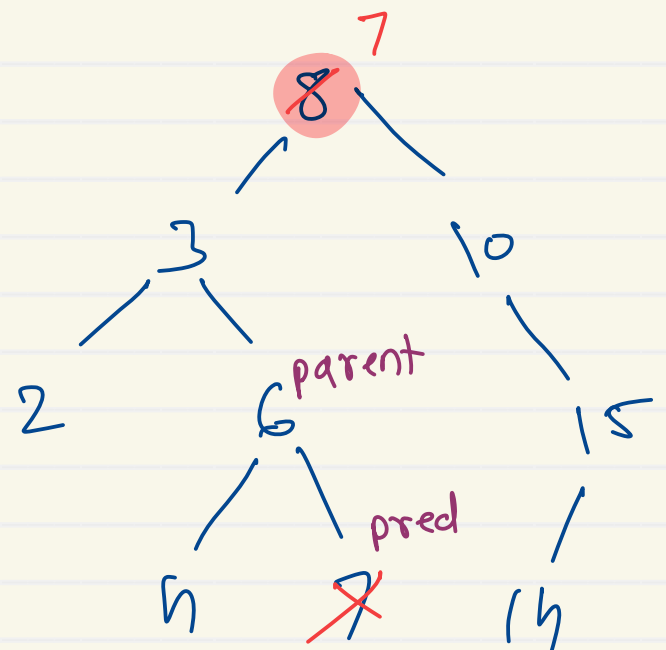
1. find predecessor of a node to be deleted
2. update value of predecessor in a node to be deleted.
3. delete predecessor.

pred	parent
3	3
6	6
7	6

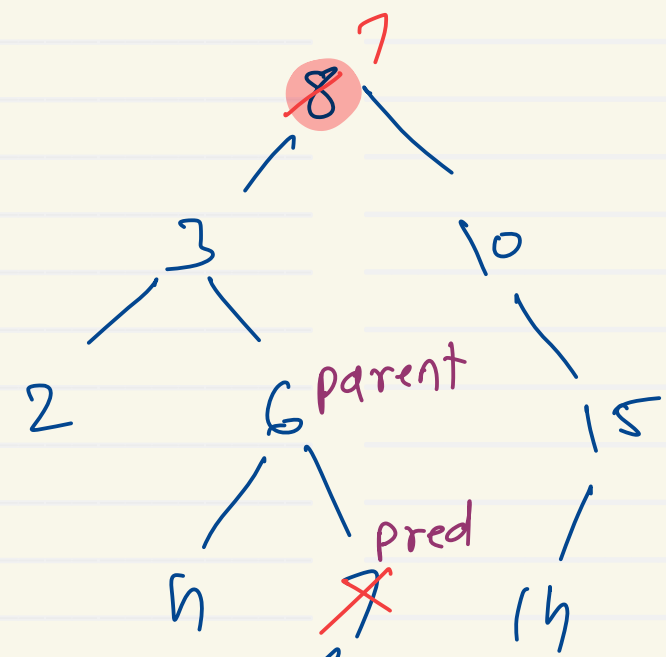
```
Node pred = trav.left;
parent = trav;
while(pred.right != null) {
    parent = pred;
    pred = pred.right;
}
```

In order : 2 3 4 6 7 8 10 14 15

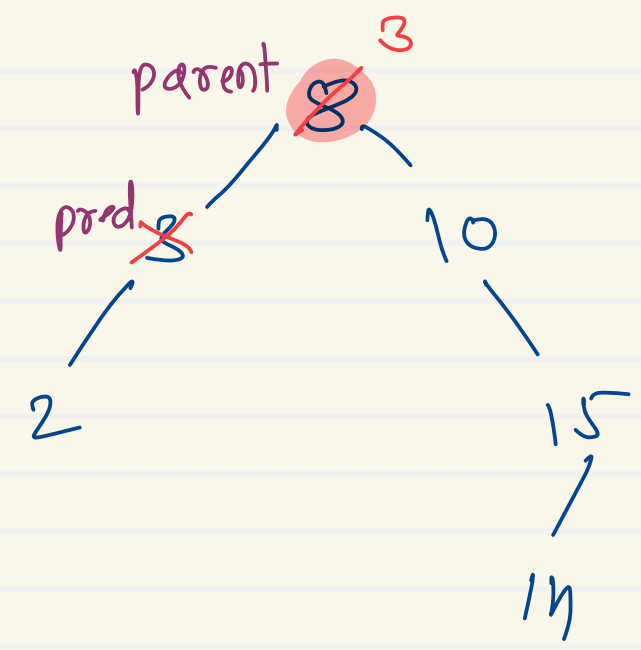
left sub tree extreme right ← inorder predecessor → inorder successor → right sub tree extreme left



if (pred == parent.right)
 parent.right = null
 ↑
 pred.left

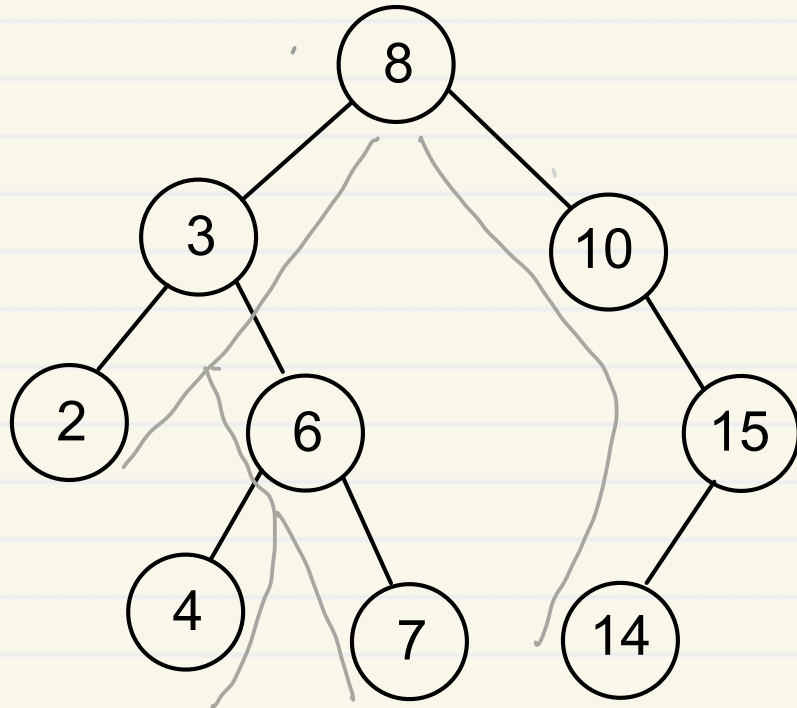


if (pred == parent.right)
 parent.right = 6.5
 ↑
 pred.left



if (pred == parent.left)
 parent.left = 2
 ↑
 pred.left

Binary Search Tree - DFS Traversal (Depth First Search)



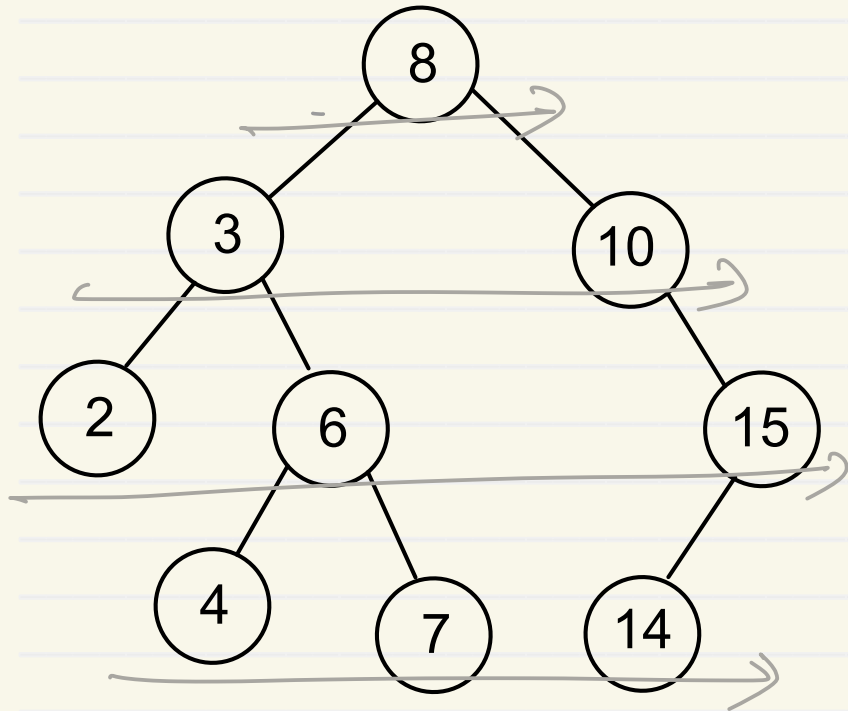
Stack

14
15
4
7
2
6
3
10
8

1. Push root node on stack
2. Pop one node from stack
3. Visit (print) popped node
4. If right exists, push it on stack
5. If left exists, push it on stack
6. While stack is not empty, repeat step 2 to 5

DFS Traversal : 8, 3, 2, 6, 4, 7, 10, 15, 14

Binary Search Tree - BFS Traversal (Breadth First Search)



Queue

8
3
10
2
6
15
4
7
14

1. Push root node on queue
2. Pop one node from queue
3. Visit (print) popped node
4. If left exists, push it on queue
5. If right exists, push it on queue
6. While queue is not empty, repeat step 2 to 5

(level order traversal)

BFS Traversal : 8, 3, 10, 2, 6, 15, 4, 7, 14